# Laboratorium 5 – Biblioteka Standardowa

Język C udostępnia zbiór podstawowych funkcji zawartych w tzw. bibliotece standardowej.
Oto kilka przydatnych plików nagłówkowych dostępnych w każdej wersji języka C:

- math.h - funkcje matematyczne, np.:
  - funkcje trygonometryczne sin(), cos(),
  - pierwiastek kwadratowy sqrt(), potęga pow()
  - logarytm naturalny log(), logarytm dziesiętny log10(), funkcja wykładnicza exp()
- ctype.h - klasyfikacja znaków, np.:
  - isupper() sprawdza, czy znak jest mała literą,
  - toupper() zamienia mała literę na dużą
- stdlib.h - zbiór podstawowych funkcji, np.:
  - rand() generator liczb losowych. Funkcja zwraca losową wartość całkowitą z zakresu od 0 do RAND_MAX
  - exit() zamknięcie programu
  - system() uruchomienie polecenia systemowego
- string.h - funkcje do manipulacji napisami, np.:
  - strcmp() porównanie kolejności alfabetycznej dwóch napisów
  - strcpy() skopiowanie napisu
- time.h - obsługa czasu, np.:
  - clock() czas wykonywania programu
  - time() czas, który upłynął od dnia 1 stycznia 1970 roku
- stdio.h - obsługa wejścia/wyjścia, np. fprintf(), fscanf()

Chcąc skorzystać z funkcji z biblioteki należy na początku pliku włączyć odpowiedni plik nagłówkowy za pomocą dyrektywy #define.

## Przykład: funkcja rand()

Funkcja rand() pochodzi z biblioteki stdlib.h i pozwala wygenerować losową wartość całkowitą.

Tak wygląda deklaracja tej funkcji:

```
int rand(void);
```

Funkcja ta przy każdym uruchomieniu zwraca liczbę całkowitą pseudolosową z zakresu od 0 do RAND_MAX. Kolejne wartości pseudolosowe powstają w sposób iteracyjny i ich kolejność zależy od początkowej wartości, tzw. ziarna. Generator liczb losowych można zainicjować dowolna liczbą całkowitą seed za pomocą funkcji srand()

```
void srand(int seed);
```

Chcąc uzyskać przy każdorazowym uruchomieniu programu inny ciąg liczb losowych można zainicjować generator wartością związaną z aktualnym czasem: funkcja time() z biblioteki time.h zwraca liczbę sekund, które upłynęły od dnia 1 stycznia 1970 roku godziny 0:00:00 czasu uniwersalnego.

Poniży program wykorzystuje funkcję rand() do przeprowadzenia symulacji wyników 10-cio krotnego rzutu kostką (losowa wartość od 1 do 6).

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int i, x;

    srand(time(0));

    for(i=0; i<10; i++)
    {
        x = rand() % 6 + 1;
        printf("%d\n", x);
    }

    return 0;
}
```

**Zadanie 1: Liczba pi**

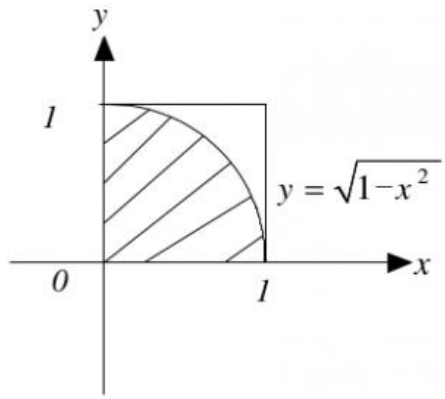
Napisz funkcję o nazwie pi, która za pomocą metody Monte Carlo wyznaczy przybliżoną wartość liczby ππ. Metoda Monte Carlo pozwala obliczyć przybliżoną wartość pola powierzchni ćwiartki koła o promieniu **r=1** za pomocą procesu losowego, w którym wykonujemy **n** losowań punktów w obszaru kwadratu o boku **1**. Przybliżona wartość pola ćwiartki koła będzie równa **k / n**, gdzie **k** to ilość trafień w tarczę koła (ćwiartki) a **n** to ilość wszystkich losowań.
Ćwiartka koła na odcinku **[0,1]** opisana jest funkcją

$$y = \sqrt{1 - x^2}$$

Z wzoru na pole koła wynika że pole ćwiartki koła o promieniu 1:

$$P = \frac{1}{4}\pi r^2 = \frac{\pi}{4}$$

$$\pi = 4 \cdot P \approx 4\frac{k}{n}$$




Algorytm:

1. ustaw k=0 (ilość trafień)
2. powtórz n razy:
    1. wylosuj współrzędne punktu (x, y), gdzie x, y ∈ [0, 1]
    2. jeżeli punkt (x, y) trafił w koło (tzn. jeśli $y<\sqrt{1+y^2}$) wówczas zwiększamy zmienną k o jeden
3. zwróć wynik 4 * k / n

Funkcja pi zdefiniowana jest zgodnie ze specyfikacją:
**Argumenty funkcji**: argumentem funkcji jest liczba całkowita n określająca ilość losowań.
**Wartość zwracana**: funkcja zwraca liczbę rzeczywistą, przybliżoną wartość liczby π

Napisz program, który korzystając z funkcji pi wyznaczy i wypisze przybliżoną wartość liczby π uzyskaną dla n=1,10,100,1000,10000,100000, 1000000 losowań.

**Przykładowy wynik działania programu**

```
       N    pi
-------------------
       1  0.000000
      10  2.800000
     100  3.000000
    1000  3.144000
   10000  3.124400
  100000  3.137120
 1000000  3.141528
10000000  3.141430
```

**Zadanie 2: Pomiar czasu**

Do programu z poprzedniego ćwiczenia (liczba pi) dodaj procedurę pomiaru czasu wykonywania procedury wyznaczania liczby π metodą Monte Carlo. Dla każdej z liczby losowań n=1,10, ... ,1000000 program wypisuje czas działania funkcji pi zmierzony za pomocą funkcji clock() z biblioteki time.h. Wynik przedstaw w sekundach, tzn. jednostki czasu określone przez funkcję clock() należy podzielić przez CLOCKS_PER_SEC.

**Przykładowy wynik działania programu**

```
       N    pi      czas [s.]
----------------------
       1  4.000000  0.000014
      10  2.800000  0.000002
     100  3.080000  0.000005
    1000  3.072000  0.000036
   10000  3.133600  0.000324
  100000  3.144280  0.003397
 1000000  3.139108  0.028748
10000000  3.140749  0.264698
```

## Tablica jako argument funkcji

Argumentem funkcji może być również tablica, wówczas definicja funkcji wygląda w następujący sposób:

```
void funkcja(int tablica[])
{
   int x;
   x = tablica[2];
}
```

Zwróć uwagę, że tablica deklarowana w argumencie (int tablica[]) nie zawiera informacji o rozmiarze. Wewnątrz funkcji nie ma informacji o tym jak dużo elementów mieści tablica. W przypadku funkcji, które mają działać dla dowolnie dużych tablic, informacja o rozmiarze musi być przekazana wartością drugiego argumentu.

Przykład definicji funkcji, która liczy sumę n elementów tablicy tab:

```
float suma(float tab[], int n)
{
   float s = 0;
   int i = 0;

   while( i < n )
   {
      s = s + tab[i];
      i = i + 1;
   }
   return s;
}
```

Wywołanie funkcji, której argumentem jest tablica nie różni się niczym od wywołania funkcji, której argumentami są zmienne typów prostych. W argumencie funkcji podajemy nazwę tablicy.

Przykład:

```
int main()
{
   float t[100], x;

   t[0] = 5;
   t[1] = 2;

   x = suma(t, 2);
}
```

Tablica, która jest argumentem funkcji, nie jest kopiowana do funkcji, tj. funkcja działa na oryginalnej tablicy i może zmodyfikować jej zawartość.

**Przykład: funkcja wczytująca liczby do tablicy**

```
#include <stdio.h>
#define MAX 100

void wczytaj(float t[], int n)
{
        int i;
        printf("\nPodaaj kolejne liczby:\n");

        for(i=0;i<n;i++)
        {
                printf("tab[%d]=",i);
                scanf("%f",&t[i]);
        }
}

float suma(float tab[], int n)
{
   float s = 0;
   int i;

   for(i=0; i < n; i++) s = s + tab[i];

   return s;
}

int main()
{
   int n;
   float tablica[MAX], x;

   printf("Podaj rozmiar tablicy: ");
   scanf("%d", &n);

   wczytaj(tablica, n);
   x = suma(tablica, n);

   printf("Suma liczb wynosi %f\n", x);

   return 0;
}
```

**Zadanie 3: Losowanie lotto**

Napisz program, który wylosuje K liczb całkowitych z zakresu od 1 do N bez zwracania. Wykorzystaj w tym celu funkcję wymieszaj() zaimplementowaną zgodnie z podaną specyfikacją.

Zdefiniuj funkcję o nazwie wymieszaj(), która przestawia w sposób losowy pozycję liczb znajdujących się w tablicy.
**Argumenty funkcji**:

- tablica z liczbami całkowitymi,
- liczba całkowita n określająca ilość liczb znajdujących się w tablicy

**Wartość zwracana**:

- brak wartości zwracanej. Efektem działania funkcji jest zmiana pozycji elementów tablicy.

Algorytm mieszania elementów w tablicy zawierającej nn elementów:

- dla każdego elementu tablicy i=0,1,2,...,n−1 wykonaj:
  - wylosuj wartość całkowitą k z zakresu od 0 do n−1 (losowa pozycja w tablicy)
  - zamień wartości elementu i-tego z elementem k-tym

Napisz program, który korzystając z funkcji wymieszaj() przeprowadzi losowanie lotto, tj. wypisze K losowych wartości z zakresu od 1 do N.
Na początku programu użytkownik podaje dwie wartości całkowite K i N.

Następnie:
1. wypełnij bęben losujący kulami ponumerowanymi od 1 do N
2. dokonaj losowania (wymieszanie zawartości bębna losującego za pomocą funkcji wymieszaj())
3. wypisz numery K kul znajdujących się na początku

**Przykład (przy każdym uruchomieniu uzyskujemy inną sekwencje)**

```
K = 6
N = 49
33
1
25
29
17
42
```